Name: Gurumanie Singh Dhiman
Section: A
University ID: 911192340

Programming Project 2 Report

Summary (10pts):

This lab was actually really fun and insightful. I was able to see first hand just how performance trade-offs of the fine-grained approach vs the coarse-grained approach. It was nice to implement this and I actually found it quite challenging at first but once I got the basic parts of the code done it started making a lot more sense. I am glad we were given weeks for this because it definitely took some time to get this done. As for the learning in this project, the medium-grained approach definitely seems to be the most "balanced" approach where it compromises on specific aspects when compared to the fine-grained approach but also gives some upside when compared to the coarse-grained approach.

```
bash-5.1$ make all
make: Warning: File 'Makefile' has modification time 21378 s in the future
gcc -pthread -Wall -g -o appserver appserver.c Bank.c
gcc -pthread -Wall -g -o appserver-coarse appserver-coarse.c Bank.c
bash-5.1$ ./appserver 4 1000 requests
CHECK 1
ID 1
TRANS 1 100000 2 100000
ID 2
TRANS 1 -10000 5 10000
ID 3
TRANS 2 -20000 4 10000 7 10000
ID 4
CHECK 1
ID 5
TRANS 2 -2000 1 1000
ID 6
TRANS 1 -10000 4 -10100 5 20100
ID 7
CHECK 4
ID 8
```

```
bash-5.1$ cat requests
1 BAL 0 TIME 1762814766.378388 1762814766.388510
2 OK TIME 1762814811.156008 1762814811.196312
3 OK TIME 1762814844.625411 1762814844.665730
4 OK TIME 1762814876.727958 1762814876.788375
5 BAL 90000 TIME 1762814886.943544 1762814886.953644
6 OK TIME 1762814908.418651 1762814908.458913
7 ISF 4 TIME 1762814938.794735 1762814938.814899
8 BAL 10000 TIME 1762814943.636295 1762814943.646400
```

6.2:
(5pts) Average runtime for each program (use the "real" time)

Test1 (appserver):

```
============== Test Summary =================

Bank program parameters: 10 worker threads, 1000 bank accounts
Output file path: test_10_1000.txt
Total number of requests generated: 1400 (400 TRANS, 1000 CHECK)
Total number of results retrived: 1400

-- Final Balances --
Expected sum of balances: 9928500
  Actual sum of balances: 9948456
Note: The expected sum is calculated by assuming all the requests are processed in sequential order in a single thread. Since we are testing with 10 threads, a small difference is acceptable

-- ISF transactions --
Expected  3 ISF requests: 166 276 311
     Got  4 ISF requests: 166 275 276 311
Note: It's acceptable to have more ISF requests than expected as long as all of the expected ISF requests are correctly recognized

-- Script Run Time --
Initial deposits (100 TRANS) took 2.0 seconds to finish, script waited 20 seconds.
Random transactions (300 TRANS) took 2.3 seconds to finish, script waited 30 seconds.

-- Request Wait Time --
Total wait time for the 400 TRANS requests: 350.278 seconds, average 0.876 seconds per request
Total wait time for the 1000 CHECK requests: 10.082 seconds, average 0.010 seconds per request


real    0m55.916s
user    0m0.029s
sys     0m0.085s
bash-5.1$
```

Test2 (appserver):

```
============== Test Summary =================

Bank program parameters: 10 worker threads, 1000 bank accounts
Output file path: test_10_1000.txt
Total number of requests generated: 1400 (400 TRANS, 1000 CHECK)
Total number of results retrived: 1400

-- Final Balances --
Expected sum of balances: 9928500
  Actual sum of balances: 9948456
Note: The expected sum is calculated by assuming all the requests are processed in sequential order in a single thread. Since we are testing with 10 threads, a small difference is acceptable

-- ISF transactions --
Expected  3 ISF requests: 166 276 311
     Got  4 ISF requests: 166 275 276 311
Note: It's acceptable to have more ISF requests than expected as long as all of the expected ISF requests are correctly recognized

-- Script Run Time --
Initial deposits (100 TRANS) took 2.0 seconds to finish, script waited 20 seconds.
Random transactions (300 TRANS) took 2.3 seconds to finish, script waited 30 seconds.

-- Request Wait Time --
Total wait time for the 400 TRANS requests: 348.965 seconds, average 0.872 seconds per request
Total wait time for the 1000 CHECK requests: 10.083 seconds, average 0.010 seconds per request


real    0m55.928s
user    0m0.018s
sys     0m0.093s
bash-5.1$
```

Test3 (appserver):

```
============== Test Summary =================

Bank program parameters: 10 worker threads, 1000 bank accounts
Output file path: test_10_1000.txt
Total number of requests generated: 1400 (400 TRANS, 1000 CHECK)
Total number of results retrived: 1400

-- Final Balances --
Expected sum of balances: 9928500
  Actual sum of balances: 9948456
Note: The expected sum is calculated by assuming all the requests are processed in sequential order in a single thread. Since we are testing with 10 threads, a small difference is acceptable

-- ISF transactions --
Expected  3 ISF requests: 166 276 311
     Got  4 ISF requests: 166 275 276 311
Note: It's acceptable to have more ISF requests than expected as long as all of the expected ISF requests are correctly recognized

-- Script Run Time --
Initial deposits (100 TRANS) took 2.0 seconds to finish, script waited 20 seconds.
Random transactions (300 TRANS) took 2.3 seconds to finish, script waited 30 seconds.

-- Request Wait Time --
Total wait time for the 400 TRANS requests: 350.424 seconds, average 0.876 seconds per request
Total wait time for the 1000 CHECK requests: 10.083 seconds, average 0.010 seconds per request


real    0m55.924s
user    0m0.028s
sys     0m0.090s
bash-5.1$
```

Test1 (appserver-coarse):

```
============== Test Summary ================

Bank program parameters: 10 worker threads, 1000 bank accounts
Output file path: test_10_1000.txt
Total number of requests generated: 1400 (400 TRANS, 1000 CHECK)
Total number of results retrived: 1400

-- Final Balances --
Expected sum of balances: 9928500
  Actual sum of balances: 9948456
Note: The expected sum is calculated by assuming all the requests are processed in sequential order in a single thread. Since we are testing with 10 threads, a small difference is acceptable

-- ISF transactions --
Expected  3 ISF requests: 166 276 311
     Got  4 ISF requests: 166 275 276 311
Note: It's acceptable to have more ISF requests than expected as long as all of the expected ISF requests are correctly recognized

-- Script Run Time --
Initial deposits (100 TRANS) took 20.2 seconds to finish, script waited 20 seconds.
Random transactions (300 TRANS) took 20.5 seconds to finish, script waited 30 seconds.

-- Request Wait Time --
Total wait time for the 400 TRANS requests: 3944.995 seconds, average 9.862 seconds per request
Total wait time for the 1000 CHECK requests: 4014.278 seconds, average 4.014 seconds per request


real    1m2.950s
user    0m0.042s
sys     0m0.106s
bash-5.1$
```

Test2 (appserver-coarse):

```
============== Test Summary ================

Bank program parameters: 10 worker threads, 1000 bank accounts
Output file path: test_10_1000.txt
Total number of requests generated: 1400 (400 TRANS, 1000 CHECK)
Total number of results retrived: 1400

-- Final Balances --
Expected sum of balances: 9928500
  Actual sum of balances: 9948456
Note: The expected sum is calculated by assuming all the requests are processed in sequential order in a single thread. Since we are testing with 10 threads, a small difference is acceptable

-- ISF transactions --
Expected  3 ISF requests: 166 276 311
     Got  4 ISF requests: 166 275 276 311
Note: It's acceptable to have more ISF requests than expected as long as all of the expected ISF requests are correctly recognized

-- Script Run Time --
Initial deposits (100 TRANS) took 20.2 seconds to finish, script waited 20 seconds.
Random transactions (300 TRANS) took 20.5 seconds to finish, script waited 30 seconds.

-- Request Wait Time --
Total wait time for the 400 TRANS requests: 3944.690 seconds, average 9.862 seconds per request
Total wait time for the 1000 CHECK requests: 4014.297 seconds, average 4.014 seconds per request


real    1m2.957s
user    0m0.049s
sys     0m0.112s
bash-5.1$
```

Test3 (appserver-coarse):

```
============== Test Summary ================

Bank program parameters: 10 worker threads, 1000 bank accounts
Output file path: test_10_1000.txt
Total number of requests generated: 1400 (400 TRANS, 1000 CHECK)
Total number of results retrived: 1400

-- Final Balances --
Expected sum of balances: 9928500
  Actual sum of balances: 9949790
Note: The expected sum is calculated by assuming all the requests are processed in sequential order in a single thread. Since we are testing with 10 threads, a small difference is acceptable

-- ISF transactions --
Expected  3 ISF requests: 166 276 311
     Got  4 ISF requests: 159 166 276 311
Note: It's acceptable to have more ISF requests than expected as long as all of the expected ISF requests are correctly recognized

-- Script Run Time --
Initial deposits (100 TRANS) took 20.2 seconds to finish, script waited 20 seconds.
Random transactions (300 TRANS) took 20.6 seconds to finish, script waited 30 seconds.

-- Request Wait Time --
Total wait time for the 400 TRANS requests: 3946.267 seconds, average 9.866 seconds per request
Total wait time for the 1000 CHECK requests: 4013.832 seconds, average 4.014 seconds per request


real    1m2.946s
user    0m0.030s
sys     0m0.121s
bash-5.1$
```

Based on the above data, the average (real) time for **appserver** is 55.923 and **appserver-coarse** is 1m2.951.

6.3:

1. **(3 pts) Which technique was faster – coarse or fine-grained locking?**

The appserver was faster, this used the fine-grained locking technique.

2. **(3 pts) Why was this technique faster?**

The fine-grained locking was faster because it allowed for parallelism when the transactions touched various different accounts. However, this also comes with more overhead as a downside.

3. **(3 pts) Are there any instances where the other technique would be faster?**

The coarse-grained locking technique could be faster in situations that involve transactions which are short, which would make using the mutex lock/unlock overhead significantly more taxing than just using a single lock. Additionally, since in our case, we have very few concurrent requests anyway, the overhead of having multiple individual mutexes outweighs the benefits we get from parallelism.

4. **(3 pts) What would happen to the performance if a lock was used for every 10 accounts? Why?**

Performance would most likely sit in the middle of our coarse-grained and fine-grained approaches if a lock was used for every 10 accounts. This is because, using one lock for every 10 accounts is a form of medium-grained locking technique and would reduce our total number of mutexes from 1000 to 100 which would reduce the overhead on memory, it would allow more parallelism than our coarse-grained approach but less than our fine-grained approach. If the answer needs to be better/worse performance then I am unsure because I think that theoretically the performance would sit in the middle of both our approaches.

5. **(3 pts) What is the optimal locking granularity (fine, coarse or medium)?**

I think the optimal locking granulatity depends on our workload. For example:

- For the best parallelism but worst overhead, we should use our fine-grained approach because this is the best for situations and workloads with the highest level of concurrency. Basically it should be used in situations that involve various different accounts.
- For the least overhead but worst concurrency, we should use our coarse-grained approach because this is the best for situations and workloads with the lowest level of concurrency. Basically it should be used in situations that involve short and simple transactions with individual accounts.
- For the balanced cases, we should use our medium-grained approach because this sits in the middle of the first two. It reduces our overhead when compared to our fine-grained approach but also allows for a decent amount of concurrency over our coarse-grained approach.